CampusBazaar

Project Synopsis

A Student-Only University Marketplace and Campus Community Platform

A full-stack application for campus commerce, trusted student interactions, and university-scoped notices.

React + Vite frontend | Spring Boot REST API | MySQL persistence

Updated to reflect the verified source, integration tests, and isolated MySQL runtime exercise.

01 · OVERVIEW

Abstract & Problem

Abstract

CampusBazaar helps university students exchange goods and useful campus information within their academic community. Students can browse, search, and filter listings; contact sellers; publish and manage their own items; and report potentially unsafe content. The platform also provides a university-scoped board for Lost, Found, and campus announcements.

The application uses a React frontend and Spring Boot REST API backed by Spring Data JPA. MySQL is the normal persistent database configuration. Spring Security protects authenticated operations with stateless JWT authentication and server-side role and ownership checks. Verification indicators, moderation reports, notifications, and an administrator dashboard provide additional trust features.

Problem

Students need affordable textbooks, electronics, furniture, and campus essentials, while other students have useful items they no longer need. General-purpose marketplaces are not limited to a university community and may make relevant listings and trusted local exchange harder to find. Lost-and-found notices and campus announcements can be fragmented across unrelated channels.

> **Purpose:** Bring student-focused buying, selling, discovery, communication, moderation, and campus notices together in one application.

Objectives & Scope

Project objectives

- Provide responsive browsing of campus listings.
- Support search by term, category, seller, price range, listing type, sorting, and page.
- Allow authenticated students to create, edit, and remove their own listings.
- Support sale, rental, exchange, and giveaway listing types.
- Persist conversations and messages, with message notifications.
- Offer a browser-persistent wishlist.
- Support listing reports and administrator moderation.
- Provide university-scoped Lost, Found, and announcement posts.
- Enforce authentication and authorization in the backend.

Student users

Students register with a university association, sign in, browse listings, view details, manage a browser-local wishlist, contact sellers, report listings, publish and manage items, and use the campus board. New registrations begin pending student verification.

Administrators

Administrators use a protected dashboard to review masked student details, update verification or account status, moderate listings, review reports, and inspect marketplace summaries. Hiding controls in the frontend is not the security boundary; backend role checks protect administrative APIs.

03 · MARKETPLACE

Discovery & Listing Lifecycle

Browse and discover

- Public listing cards and product detail pages.
- Database-backed categories and available listings.
- Text, category, and seller search.
- Category, listing type, and minimum/maximum price filters.
- Newest, lowest-price, and highest-price sorting.
- Server-side pagination with bounded page size.

Publish and manage

- Signed-in students create listings using active database categories.
- Listings support sale, rent, exchange, and giveaway.
- Sale and rental require positive prices; exchange and giveaway use zero price.
- Owners may edit their listings and soft-delete them; removed rows remain stored but are excluded from public available listings.
- Administrators can separately moderate listings.

> These lifecycle rules are enforced by backend services and authorization checks, not only by frontend form validation.

04 · STUDENT INTERACTIONS

Accounts, Wishlist & Chat

Accounts & trust

Student registration and login use the backend authentication API. JWT bearer tokens protect signed-in operations. Account status and listing ownership are checked server-side. Verified-student indicators can be displayed with seller information.

Wishlist

Users can add and remove listing IDs. Current wishlist state is saved in browser local storage and persists across refreshes in that browser; it is not synchronized across accounts or devices.

Messaging

Conversations are associated with a listing and buyer/seller participants. Only participants may read or send messages. Messages persist in the configured database.

Notifications

Message notifications are persisted. The frontend refreshes messages and notifications periodically. Live WebSocket delivery is not currently implemented.

Privacy-conscious responses

User-facing API responses do not return password hashes or full student IDs. Public seller information can show verification status without exposing sensitive identity data.

05 · COMMUNITY & MODERATION

Reports & Campus Board

Reporting and administration

- Signed-in users can report listings.
- Reports are reserved for administrators, not exposed in public listing responses.
- The protected admin dashboard supports account actions, listing moderation, report review, and marketplace summary counts.
- Suspended accounts are rejected on authenticated requests.

Campus board

- Signed-in students view posts scoped to their account's university.
- Post types include Lost item, Found item, and Campus announcement.
- Students can create posts, resolve their own open posts, and remove their own posts.
- Responses omit direct contact details and student IDs.

Principal data entities

User and **University** represent accounts, roles, verification, and campus membership. **Category** organizes listings. **Listing** stores description, price, condition, type, status, category, and seller. **Conversation**, **ChatMessage**, and **UserNotification** persist communication. **ListingReport** tracks reports and review. **CampusPost** stores university-scoped notices and status.

06 · ENGINEERING

Technology & Architecture

Technology stack

React | Vite | JavaScript | Tailwind CSS | Redux Toolkit | Axios | React Router | Java 17 | Spring Boot 3 | Spring MVC | Spring Data JPA | Hibernate | Spring Security | JWT | MySQL

Request and persistence flow

1. The React single-page application sends HTTP requests to the backend API.
2. Spring Boot controllers handle request validation and endpoint security.
3. Services apply marketplace, account, messaging, moderation, and campus-board rules.
4. Spring Data repositories read and persist relational entities with JPA.
5. MySQL provides configured persistent storage; automated integration tests use isolated in-memory H2.

The frontend development server proxies `/api` requests to Spring Boot on port 8080. JWT signing material belongs in backend environment configuration, never frontend configuration or source control. Local demo administrator credentials are development-only.

07 · VERIFICATION

Tests & MySQL Runtime

Automated checks

- Backend `mvn clean package`: **13 integration tests passed** with H2.
- Frontend `npm test -- --run`: **20 tests passed**.
- Frontend `npm run build`: production build succeeded.
- H2 runtime checks confirmed public reads, HTTP 401 for unauthenticated listing creation, authenticated listing lifecycle, and campus-post operations.

Isolated MySQL 8.4.9 exercise

A disposable MySQL server bound to loopback on port 3307 used a newly created temporary data directory. The updated backend connected and started on port 8081, created schema and demo rows, and returned HTTP 200 for health, categories, listings, and paginated search.

- Listing creation and detail read-back worked; owner update persisted.
- Another user's update was rejected with HTTP 403.
- Soft deletion persisted status `REMOVED` and hid the listing from public detail.
- Campus posts were visible within their university, not to a second university.
- After backend restart, listing and campus-board records remained persisted.

Database Scope & Limitations

> **Important verification boundary:** This was a real MySQL compatibility and persistence test, but against a newly initialized disposable database—not the user's pre-existing CampusBazaar database.

No pre-existing CampusBazaar MySQL data directory, service, or application database was configured on the machine; port 3306 refused connections and no MySQL/MariaDB Windows service was registered. No pre-existing records were accessed or modified. The temporary server, test backend, and temporary data directory were stopped and removed after the test.

A persistent MySQL service and its connection credentials must be configured before using the application's normal port 8080 setup. Any existing deployment database and its records still require separate verification.

Current limitations

- Listing image selection offers a local preview; file upload and durable image storage are not implemented.
- Wishlist state is browser-local, not account-synchronized.
- Message and notification updates use polling rather than WebSockets.
- Campus-board posts do not support attachments or direct replies.

Security boundary

Public reads are separated from authenticated mutations. Backend checks enforce listing ownership, campus-board university scope, and administrator role. Passwords are hashed; full student IDs are masked in user-facing responses.

09 · NEXT STEPS

Enhancements & Conclusion

Future enhancements

- Add secure image upload and durable image storage.
- Persist wishlists per account and synchronize between devices.
- Add WebSocket-based messaging and notification delivery.
- Complete deployment-database verification and production configuration review.
- Extend profile and student-verification workflows.
- Build out rental requests, exchange proposals, services, and deal/coupon workflows.

Conclusion

CampusBazaar provides an integrated foundation for a university marketplace and student community board. Database-backed listings, authenticated ownership actions, server-side search and pagination, student trust signals, participant-scoped communication, report moderation, and university-scoped notices work together in one application.

Automated tests, an isolated H2 runtime, and a disposable MySQL 8.4 runtime validate the updated flows. Verification against a pre-existing CampusBazaar database and deployed credentials remains outstanding.

> **Result:** Verified application changes without replacing backend integration with mock data or altering any pre-existing database records.

Generated from the project synopsis. Test and runtime statements reflect the completed checks documented in the source synopsis.